

MedVision Thesis

Code Walkthrough

A detailed walkthrough of the codebase: architecture, key functions, data flow, and the audit pipeline.

This document assumes you have read the Project Guide. It covers the HOW, not the WHAT.

1. Notebook Structure

The project consists of 11 training notebooks + 1 audit notebook. Each notebook shares the same cell structure (cells 0-7 are identical across Parts 2-11), differing only in cell 8 (the training driver) and cell 9+ (analysis/save).

0	Markdown	Title + setup instructions	Part number, estimated time, inputs to attach, resume instructions
1	Code	Install dependencies	!pip install transformers, datasets, torchvision, torchxrayvision, etc.
2	Code	Imports	import torch, numpy, pandas, sklearn, transformers, torchxrayvision, etc. + mem_stats() helper
3	Code	Configuration	class Config: SEED_LIST, BASELINE_EPOCHS, LR_*, BATCH_SIZE, NORMAL_CAP, etc. + set_seed()
4	Code	Load Part 1 output	Load mimic_df.pkl, final_split.pkl, verify images, copy checkpoints, load all_results.pkl for resume
5	Code	Datasets + transforms	ChestXrayDataset, ImageOnlyDataset, TextOnlyDataset, train_transform, val_transform, build_loaders()
6	Code	Model architecture	XRVImageEncoder, TextEncoder, DualPathCADQ, LearnedMixinFusion, NoiseConsistencyFusion, AdaptiveCADQ, ChestXrayVLM, FocalLoss, ImageOnlyModel, TextOnlyModel
7	Code	Training + eval + OOF	evaluate_model(), train_vlm(), OOF TF-IDF bias model, gacr_b_loss(), p_bias_lookup, gacr_lookup
8	Code	Training driver (varies per Part)	The actual training loop — which seeds, which variant, which epochs
9	Code	Analysis (Part 11 / audit only)	ICM, TOST, gate extraction, figures, stats, compliance checklist, final save
10	Code	Final save	Save all_results.pkl, img_text_results.pkl, print completion message

2. Cell 3: Configuration (Config class)

The Config class holds ALL hyperparameters. It is defined once in cell 3 and used by every subsequent cell. Changing a value here affects all training.

Key parameters:

```
class Config:
    SEED = 42 # default seed for baselines
    SEED_LIST = [42, 123, 456, 789, 1010, 1111, 1212, 1313] # 8 seeds for main experiments
    ABLATION_SEED_LIST = [42, 123, 456] # 3 seeds for ablations (unused in final)
    BASELINE_EPOCHS = 10 # epochs for all variants
    NORMAL_CAP = 2500 # max Normal images
    PNEUMONIA_CAP = 2500 # max Pneumonia images
    PNEUMOTHORAX_CAP = 1000 # max Pneumothorax images (rare class)
    BATCH_SIZE = 8 # batch size (small for memory)
    GRAD_ACCUM_STEPS = 4 # gradient accumulation (effective batch = 32)
    LR_IMG_BACKBONE = 1e-5 # DenseNet LR
    LR_TEXT = 1e-5 # Bio_ClinicalBERT LR
    LR_FUSION = 1e-4 # Cross-attention LR
    LR_GATE = 5e-5 # Gate LR
    CADQ_MIN_IMAGE_GATE = 0.20 # gate minimum bound
    CADQ_MAX_IMAGE_GATE = 0.80 # gate maximum bound
    GATE_INIT_LOGITS = [-0.5, 0.0, 0.5] # init -> alpha = [0.4265, 0.5000, 0.5735]
    LMH_G_MAX = 5.0 # LMH gate cap
    LMH_LAMBDA_ENTROPY = 0.05 # LMH entropy penalty weight
    LMH_WARM_START = True # LMH warm-starts from Baseline checkpoint
    NOISE_CONSISTENCY_LAMBDA = 0.3 # NC loss weight
    FOCAL_GAMMA = 1.5 # focal loss gamma
    LABEL_SMOOTHING = 0.05 # label smoothing
    STOCHASTIC_TEXT_DROP = 0.10 # 10% of training, drop text entirely
```

3. Cell 6: Model Architecture

All 5 fusion variants share the same encoders but differ in the fusion layer. Here is the class hierarchy.

3.1 Image Encoder (XRVImageEncoder)

Uses TorchXRayVision's DenseNet-121 pretrained on MIMIC-CXR. The convolutional stem (conv0, norm0, pool0, denseblock1, transition1) is FROZEN to save memory. A projection head (Linear -> LayerNorm -> GELU -> Dropout) maps the 1024-dim DenseNet feature to a 512-dim projection.

```
class XRVImageEncoder(nn.Module):
    def __init__(self, proj_dim=512, freeze_stem=True):
        self.backbone = xrv.models.DenseNet(weights='densenet121-res224-mimic_ch')
        # Freeze stem layers
        if freeze_stem:
            for name, p in self.backbone.named_parameters():
                if name.startswith(('features.conv0', 'features.norm0', 'features.pool0',
                                    'features.denseblock1', 'features.transition1')): p.requires_grad = False
        self.projection = nn.Sequential(Linear(1024, 512), LayerNorm(512), GELU(), Dropout(0.25))
    def forward(self, x): return self.projection(self.backbone.features2(x))
```

3.2 Text Encoder (TextEncoder)

Uses Bio_ClinicalBERT (emilyalsentzer/Bio_ClinicalBERT). The first 4 transformer layers are frozen. The [CLS] token output is projected to 512-dim.

```
class TextEncoder(nn.Module):
    def __init__(self, model_name='emilyalsentzer/Bio_ClinicalBERT', proj_dim=512,
                 frozen_layers=4):
        self.bert = AutoModel.from_pretrained(model_name)
        for i in range(min(frozen_layers, len(self.bert.encoder.layer))):
            for p in self.bert.encoder.layer[i].parameters(): p.requires_grad = False
        self.projection = nn.Sequential(Linear(768, 512), LayerNorm(512), GELU(), Dropout(0.25))
    def forward(self, input_ids, attention_mask):
        return self.projection(self.bert(input_ids, attention_mask).last_hidden_state[:, 0, :])
```

3.3 Fusion Variant 1: Baseline (DualPathCADQ)

Cross-attention (image queries text) followed by a STATIC per-class gate. The gate is a single nn.Parameter of shape [3] (one gate per class), initialized at logits [-0.5, 0.0, 0.5].

```
class DualPathCADQ(nn.Module):
    def __init__(self, proj_dim=512, num_classes=3, num_heads=8, dropout=0.25,
                 min_gate=0.20, max_gate=0.80, gate_init_logits=None):
        self.cross_attn = nn.MultiheadAttention(proj_dim, num_heads, dropout, batch_first=True)
        self.norm = nn.LayerNorm(proj_dim)
        self.img_head = nn.Linear(proj_dim, num_classes)
        self.text_head = nn.Linear(proj_dim, num_classes)
        self.gate_logits = nn.Parameter(torch.tensor(gate_init_logits or [-0.5, 0.0, 0.5]))
    def get_alpha(self): return 0.20 + 0.60 * torch.sigmoid(self.gate_logits)
    def forward(self, img_feat, text_feat, p_bias=None):
        attn_out, _ = self.cross_attn(query=img_feat.unsqueeze(1), key=text_feat.unsqueeze(1),
                                       value=text_feat.unsqueeze(1))
        img_enriched = self.norm(attn_out.squeeze(1) + img_feat)
        alpha = self.get_alpha() # shape [3], static per class
        logits = alpha * img_head(img_enriched) + (1-alpha) * text_head(text_feat)
        return logits, alpha
```

KEY POINT: alpha is input-INDEPENDENT. It learns one global mixing ratio per class, not per-example. The gate stagnation finding (alpha frozen at init across all 8 seeds) shows this gate never received a useful gradient signal.

3.4 Fusion Variant 2: LMH (LearnedMixinFusion, PoE)

Fixed 50/50 fusion PLUS a learned input-dependent gate that injects the text-bias correction. Warm-started from the corresponding Baseline checkpoint.

```
class LearnedMixinFusion(nn.Module):
    # Main logits: 0.5 * img_head + 0.5 * text_head (FIXED, no gate)
    # PLUS: log_softmax(main) + g * log_p_bias
    # where g = 5.0 * sigmoid(g_head([img_enriched, text_feat])) (input-DEPENDENT)
    # and log_p_bias = precomputed OOF TF-IDF text-only baseline (temperature-calibrated, smoothed)
    def forward(self, img_feat, text_feat, log_p_bias=None):
        main_logits = 0.5 * img_head(img_enriched) + 0.5 * text_head(text_feat)
        if log_p_bias is not None:
            h = torch.cat([img_enriched, text_feat], dim=-1).detach()
            g = 5.0 * sigmoid(g_head(h)).squeeze(-1)
            return log_softmax(main_logits) + g * log_p_bias, g
        return main_logits, torch.zeros(...)
```

KEY POINT: LMH is the PoE implementation. It subtracts the text-bias by adding $\log p_{\text{bias}}$ (which is $\log P_{\text{bias}}(y|x_{\text{text}})$ from the TF-IDF baseline) scaled by a learned gate g . Under circularity ($y = f(x_{\text{text}})$), P_{bias} is a near-perfect surrogate for y , so this subtraction is incoherent. The training instability (val F1 crashes after epoch 2-3) is the empirical signature of this incoherence.

3.5 Fusion Variant 3: NoiseConsistencyFusion

Fixed 50/50 fusion (no gate, no text-bias). The intervention is in the LOSS, not the architecture: an anti-consistency loss that MAXIMIZES KL divergence between (image, text) and (noised_image, text) predictions.

```
class NoiseConsistencyFusion(nn.Module):
    def forward(self, img_feat, text_feat):
        return 0.5 * img_head(img_enriched) + 0.5 * text_head(text_feat), torch.zeros(...)
    def compute_consistency_loss(self, model, images, input_ids, attention_mask, device):
        logits_real, _ = model(images, input_ids, attention_mask)
        p_real = softmax(logits_real)
        noise = torch.randn_like(images) * 0.25
        logits_noised, _ = model(images + noise, input_ids, attention_mask)
        p_noised = softmax(logits_noised)
        kl = F.kl_div(log(p_noised + 1e-8), p_real, reduction='none').sum(dim=-1)
        return -kl.mean() # NEGATIVE KL = maximize divergence
```

KEY POINT: This is ANTI-consistency, not naive consistency. The model is forced to be SENSITIVE to image noise (maximize prediction change when image is noised). But under circularity, the model can achieve this by learning noise-correlated features in the text pathway, bypassing the actual image content.

3.6 Fusion Variant 4: AdaptiveCADQ

Dynamic entropy-gated fusion. The gate α is computed by a 2-layer MLP from the ENTROPY of each modality's prediction. If the image head is uncertain (high entropy), downweight the image.

```
class AdaptiveCADQ(nn.Module):
    def __init__(self, ...):
        self.gate_mlp = nn.Sequential(Linear(2, 16), GELU(), Dropout, Linear(16, 1), Sigmoid())
    def get_alpha(self, img_logits, text_logits):
        pi = softmax(img_logits); pt = softmax(text_logits)
        ei = entropy(pi); et = entropy(pt) # entropy of each head's prediction
        return self.gate_mlp(cat([ei, et])).squeeze(-1) # input-DEPENDENT
    def forward(self, img_feat, text_feat):
        il = img_head(img_enriched); tl = text_head(text_feat)
        a = get_alpha(il, tl)
        return a * il + (1-a) * tl, a
```

3.7 Fusion Variant 5: GACR-B (in the training driver, not a separate class)

GACR-B uses the SAME Baseline CADQ architecture but adds an auxiliary loss. The loss is defined inline in cell 8:

```
def gacr_b_extra(logits, alpha, images, input_ids, attention_mask, labels, model, indices=None,
**kw):
    if indices is None: return torch.tensor(0.0, device=logits.device)
    tb_preds, tb_probs = gacr_lookup.get(indices.cpu().numpy())
    return gacr_b_loss(logits, labels, torch.from_numpy(tb_preds).long().to(logits.device),
tb_probs)

def gacr_b_loss(logits, labels, text_baseline_preds, text_baseline_probs):
    p_full = F.softmax(logits, dim=-1)
    tb_soft = torch.from_numpy(text_baseline_probs).float()
    sim = F.cosine_similarity(p_full, tb_soft, dim=-1)
    mask = (text_baseline_preds != labels).float() # 1 where text baseline is WRONG
    return (mask * sim).mean() # penalize agreement with WRONG text baseline
```

KEY POINT: The loss penalizes cosine similarity between the model's prediction and the text baseline ONLY on examples where the text baseline is wrong. This is the corrected sign (the v10 bug had $(\text{mask} * (1 - \text{sim})).\text{mean}()$ which rewarded agreement with the wrong baseline). GACR-B showed the largest improvement of any variant (ICM = 0.0037) but still failed to reach significance ($p = 0.055$) and its ICM was still equivalent to zero (TOST $p = 0.024$).

4. Cell 7: Training Function (train_vlm)

The `train_vlm()` function is the core training loop. It handles warm-starting, gradient accumulation, mixed precision, OneCycleLR, best-val checkpoint selection, and all variant-specific loss logic.

Key flow:

1. Warm-start (if `warm_start_path`): Load Baseline checkpoint, copy matching params
2. Parameter groups: Separate LR for image backbone, text, fusion, gate, `g_head`
3. Optimizer: AdamW with `weight_decay=1e-4`
4. Scheduler: OneCycleLR with 10% warmup, `total_steps = len(train) * epochs / grad_accum`
5. Criterion: FocalLoss(`alpha=CLASS_WEIGHTS`, `gamma=1.5`, `label_smoothing=0.05`)
6. Training loop (per epoch):
 - a. For each batch: forward -> compute loss -> backward -> step (every 4 batches)
 - b. If LMH: `loss = CE + lambda * entropy_penalty(g, log_p_bias)`
 - c. If GACR-B: `loss = CE + 0.3 * gacr_b_extra(logits, ...)`
 - d. If NC: `loss = CE + 0.3 * compute_consistency_loss(...)` (after warmup)
 - e. Else: `loss = CE`
 - f. Evaluate on val -> track `best_val_score`
7. Load best-val checkpoint -> return model, history

The `evaluate_model()` function

Computes `accuracy`, `macro_f1`, `per_class_f1`, `per_class_recall`, `AUC (ovr macro)`, `confusion_matrix`, `classification_report`, `all_preds`, `all_labels`, `all_probs`. Used for both val and test evaluation.

5. Cell 4: Data Loading + Resume Logic

Cell 4 loads the Part 1 output (mimic_df.pkl, final_split.pkl, images) and sets up the resume mechanism.

Resume logic (critical for multi-session training):

```
# 1. Search /kaggle/input/ for all_results.pkl from previous Parts
# 2. Copy it to /kaggle/working/all_results.pkl
# 3. Load it into all_results dict
# 4. In cell 8 training loop: if f'{variant}_s{seed}' in all_results: skip
# 5. After each seed: pickle.dump(all_results, RESUME_PATH)

# Also copies .pt checkpoints from /kaggle/input/ to Config.SAVE_DIR
# This enables LMH warm-start (LMH needs the corresponding baseline_s{seed}.pt)
```

KEY POINT: The resume mechanism is CUMULATIVE. Each Part's Kaggle output contains all_results.pkl with ALL previous results + checkpoints. When you attach Part N's output as input to Part N+1, cell 4 finds the pickle, copies it forward, and the training loop skips already-done seeds.

6. Audit Notebook: ICM + Gate Extraction (Cell 9)

The audit notebook (medvision-q1-audit-v2.ipynb) is analysis-only. Cell 9 contains the Q1 audit code.

6.1 ICM Computation

```
gray_f1 = gray_results.get('macro_f1', 0.9381) # from img_text_results.pkl
for variant in ['baseline', 'lmh', 'noise_consistency', 'adaptive_cadq']:
    for seed in Config.SEED_LIST:
        vlm_f1 = all_results[f'{variant}_s{seed}']['test']['macro_f1']
        icm = (vlm_f1 - gray_f1) / vlm_f1
        icm_data.append({'Variant': variant, 'Seed': seed, 'VLM_F1': vlm_f1, 'ICM': icm})
```

6.2 TOST Equivalence Test

```
from scipy.stats import ttest_lsamp
equiv_bound = 0.01 # +/- 1% relative F1
t_low, p_low = ttest_lsamp(icms, -equiv_bound)
t_high, p_high = ttest_lsamp(icms, equiv_bound)
p_tost = max(p_low, p_high) # TOST p-value
# If p_tost < 0.05: ICM is statistically equivalent to zero
```

6.3 Gate Weight Extraction

```
for seed in Config.SEED_LIST:
    ckpt_path = os.path.join(Config.SAVE_DIR, f'baseline_s{seed}.pt')
    state = torch.load(ckpt_path, map_location='cpu')
    gate_logits = state.get('fusion.gate_logits', None) # nn.Parameter
    alpha = 0.20 + 0.60 * torch.sigmoid(gate_logits) # [3] values
    gate_data.append({'Seed': seed,
                     'Alpha_Normal': alpha[0].item(),
                     'Alpha_Pneumonia': alpha[1].item(),
                     'Alpha_PNX': alpha[2].item()})

# Compute drift from init
init_mean_alpha = 0.5000 # from gate_logits = [-0.5, 0.0, 0.5]
drift = abs(mean_alpha - init_mean_alpha)
if drift < 0.005: print('GATE STAGNATION')
```

7. Data Flow (End-to-End)

The complete data flow from raw MIMIC-CXR to final paper:

1	PhysioNet (raw)	Part 1: Download metadata, build cohort (2500/2500/1000), patient-level split	mimic_df.pkl, final_split.pkl, images/
2	Part 1 output	Parts 2-9: Train 4 variants $\times$ 8 seeds = 32 runs	all_results.pkl (32 entries), 32 .pt checkpoints
3	Part 1 + Parts 2-9	Part 10: Train GACR-B $\times$ 8 seeds (corrected loss)	all_results.pkl (40 entries), 8 GACR-B .pt checkpoints
4	Part 1 + Part 10	Part 2: Train 3 baselines (Image-Only, Text-Only, Gray-Square)	img_text_results.pkl (3 entries)
5	all_results.pkl + img_text_results.pkl + .pt files	Audit notebook: Compute ICM, TOST, gate extraction, figures	q1_audit_results.json, 7 figures, medvision_final_results.json
6	Audit outputs	Write paper (manual)	MedVision_Q1_Paper_FINAL.docx

End of Code Walkthrough. See Run Instructions PDF for step-by-step reproduction.